



Football Analysis Platform — Technical Book & Architecture Manual

A complete, annotated technical guide to the architecture, business logic, design patterns, mathematical models, live in-match engine, and core code implementations of the Football RAG + GNN SaaS system.

Repository: [aliosama8399/football-analysis](https://github.com/aliosama8399/football-analysis)

Version: 2.0 (Live In-Match Module Included)

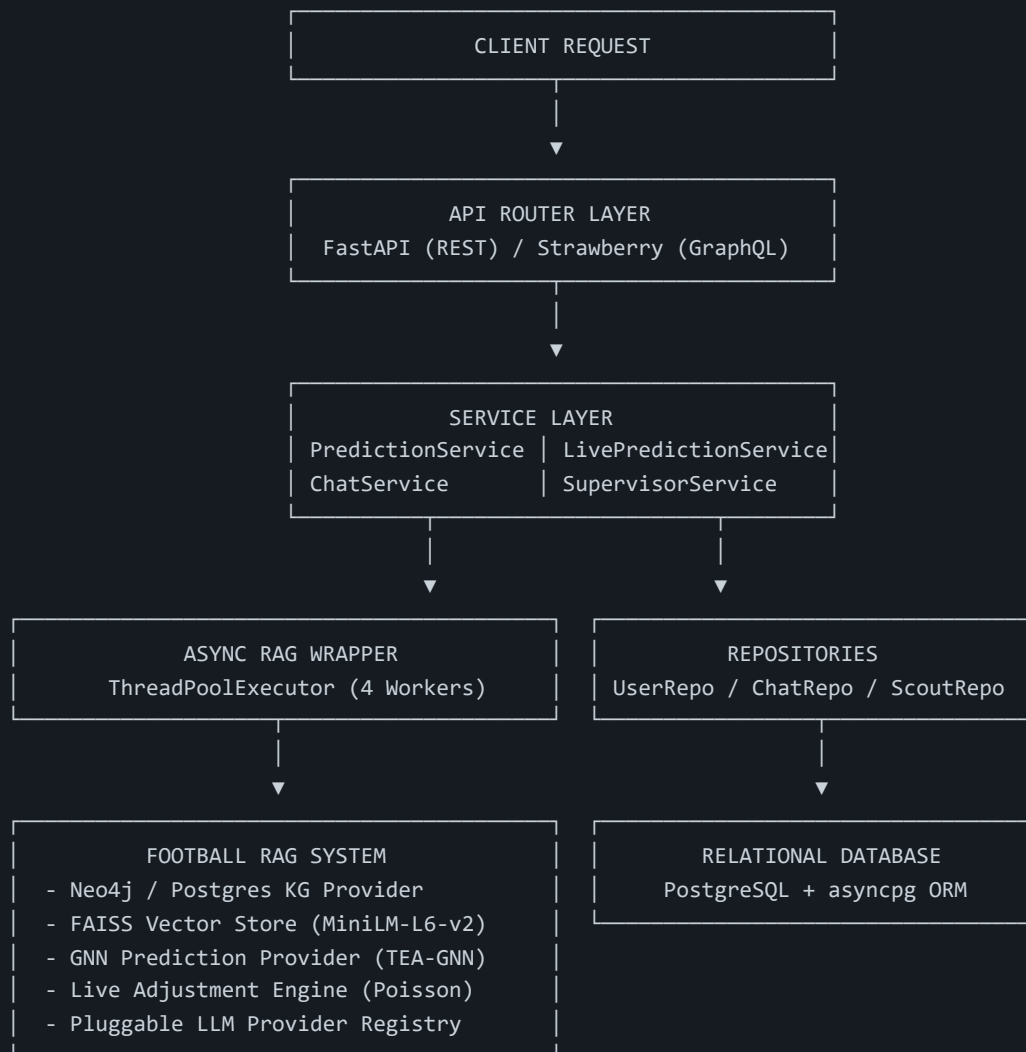
Table of Contents

1. System Overview & High-Level Architecture
2. Technology Stack & Core Dependencies
3. Data Pipeline & Preprocessing Engine
4. 3.1 Data Sources & Collectors
5. 3.2 Team Name Standardization Registry
6. 3.3 Feature Engineering & Memory-Safe Preprocessing
7. Graph Construction — FootballGraphBuilder
8. Machine Learning & GNN Architectures
9. 5.1 Edge Classification Formulation
10. 5.2 Shared EdgeClassifier MLP Head
11. 5.3 Baseline GNN Architectures
12. 5.4 The Novel Architecture: TEA-GNN
13. 5.5 Model Architectural Comparison
14. Live In-Match Prediction Module (New)
15. 6.1 Module Architecture & Data Flow
16. 6.2 Pure-Math Engine: Poisson Inversion & Conditioning
17. 6.3 Live Pace Multipliers & Driver Extraction
18. 6.4 Live Prediction Service
19. 6.5 REST API Endpoint & Schemas
20. RAG System & Two-Expert Architecture
21. 7.1 FootballRAGSystem Orchestrator
22. 7.2 The Two-Expert Ensemble Design
23. Knowledge Graph & Vector Providers

- 24. 8.1 Knowledge Graph Providers (Neo4j & Postgres)
 - 25. 8.2 Vector Provider (FAISS + MiniLM)
 - 26. LLM Providers & Extensible Registry
 - 27. API Infrastructure & Asynchronous Bridge
 - 10.1 Application Lifespan & Bootstrap
 - 10.2 Dependency Injection Container
 - 10.3 AsyncRAGWrapper Thread-Pool Bridge
 - 10.4 JWT Authentication & Guard Dependencies
 - 28. Business Logic & Service Layer
 - 11.1 PredictionService
 - 11.2 ChatService & Conversational Memory
 - 11.3 SupervisorService & Feature-Complete Match Ingestion
 - 29. Repositories & Data Access Layer
 - 12.1 ChatRepository & FeedbackRepository
 - 12.2 ScoutRepository & Multi-Provider Player Scouting
 - 30. Database Schema & Migrations
 - 31. Design Patterns & Engineering Principles
 - 32. Concurrency, Performance & Gotchas
-

1. System Overview & High-Level Architecture

The Football Analysis SaaS Platform predicts pre-match and live in-match football outcomes while serving tactical explanations through a two-expert system: - **Expert 1 (Quantitative GNN / Math Engine):** TEA-GNN pre-match probabilities + Poisson-conditioned live math engine. - **Expert 2 (Qualitative Fine-Tuned LLM):** Qwen3/SmolLM2 fine-tuned model providing grounded, coach-actionable tactical explanations.



2. Technology Stack & Core Dependencies

Category	Technology	Version	Purpose
API Framework	FastAPI	0.140.0	High-performance ASGI REST engine
GraphQL Layer	Strawberry-GraphQL	0.323.2	Schema-first GraphQL interface
ORM & Drivers	SQLAlchemy + asyncpg	2.0.51 / 0.31.0	Asynchronous relational data access
DB Migrations	Alembic	1.18.5	Version-controlled schema migrations
Security	python-jose + bcrypt	3.5.0 / 5.0.0	JWT auth & 72-byte safe password hashing
Graph Database	Neo4j Python Driver	6.2.0	Cypher graph queries for team/match topology
Vector Store	FAISS (cpu)	1.14.3	Inner-product similarity search over embeddings
Text Embeddings	Sentence-Transformers	5.6.1	<code>all-MiniLM-L6-v2</code> dense vector representations
GNN Framework	PyTorch Geometric (PyG)	2.8.0	Temporal edge-attention graph deep learning
Deep Learning	PyTorch	2.13.0	Core tensor operations and CUDA execution
Traditional ML	scikit-learn / XGBoost	1.9.0	Tabular baseline models & scaling utilities
Data Processing	pandas + numpy	3.0.5 / 2.5.1	Vectorized feature computation & statistics
Local LLM	HuggingFace Transformers	5.14.1	Local Qwen3 / SmolLM2 fine-tuned model inference

3. Data Pipeline & Preprocessing Engine

3.1 Data Sources & Collectors

The platform aggregates multi-source raw match and squad data: 1. **football-data.co.uk**: Results, shots, corners, fouls, cards, and referees across top 5 European leagues. 2. **Understat**: Match-level expected goals (\$xG\$, \$xGA\$). 3. **Open-Meteo Weather API**: Hourly stadium temperature, precipitation, and wind speed. 4. **FBRef / Champions League**: Player-level positional statistics and squad metrics.

3.2 Team Name Standardization Registry

`data/team_registry.py` implements league-scoped team name canonicalization to solve cross-source naming discrepancies.

```

# data/team_registry.py
import re
import unicodedata
from typing import Dict, Optional

# Team alias mapping per league code (E0=EPL, SP1=La Liga, D1=Bundesliga, I1=Serie A, F1=
TEAM_ALIAS_MAP: Dict[str, Dict[str, str]] = {
    "E0": {
        "man utd": "Manchester United", "manchester utd": "Manchester United", "mancheste
        "man city": "Manchester City", "manchester city": "Manchester City",
        "spurs": "Tottenham", "tottenham hotspur": "Tottenham", "tottenham": "Tottenham",
        "wolves": "Wolverhampton", "wolverhampton wanderers": "Wolverhampton",
        "newcastle utd": "Newcastle", "newcastle united": "Newcastle", "newcastle": "Newc
        "west ham united": "West Ham", "west ham utd": "West Ham", "west ham": "West Ham"
    },
    "SP1": {
        "atletico": "Atletico Madrid", "atlético madrid": "Atletico Madrid", "atlético":
        "real madrid": "Real Madrid", "barcelona": "Barcelona", "barca": "Barcelona",
        "athletic": "Athletic Club", "athletic bilbao": "Athletic Club",
    },
    "D1": {
        "bayern": "Bayern Munich", "bayern münchen": "Bayern Munich", "bayern munich": "B
        "dortmund": "Borussia Dortmund", "bvb": "Borussia Dortmund",
        "leverkusen": "Bayer Leverkusen", "bayer leverkusen": "Bayer Leverkusen",
    },
    "I1": {
        "inter": "Inter", "inter milan": "Inter", "internazionale": "Inter",
        "ac milan": "Milan", "milan": "Milan",
        "juve": "Juventus", "juventus": "Juventus",
    },
    "F1": {
        "psg": "Paris Saint Germain", "paris sg": "Paris Saint Germain", "paris saint-ger
        "om": "Marseille", "olympic marseille": "Marseille",
    }
}

def normalize_text(text: str) -> str:
    """Strip accents and lower case for match-key lookups."""
    s = unicodedata.normalize("NFKD", str(text))
    s = "".join(c for c in s if not unicodedata.combining(c))
    return re.sub(r"\s+", " ", s).strip().lower()

def normalize_team_name(name: str, league_code: Optional[str] = None) -> str:
    """
    Standardize a team name using league-scoped alias dictionary with fallback.
    Returns original name if unmapped (fail-open strategy).
    """
    clean = normalize_text(name)
    if league_code and league_code in TEAM_ALIAS_MAP:
        if clean in TEAM_ALIAS_MAP[league_code]:
            return TEAM_ALIAS_MAP[league_code][clean]

    # Cross-league fallback search
    for lg_code, mapping in TEAM_ALIAS_MAP.items():
        if clean in mapping:
            return mapping[clean]

```

```
return name.strip()
```

3.3 Feature Engineering & Memory-Safe Preprocessing

`data/preprocess.py` processes raw match CSVs using vectorized pandas `groupby` and strict temporal shifting (`shift(1)`) to compute 80 ML features without data leakage.

```

# Core of data/preprocess.py
import pandas as pd
import numpy as np
from pathlib import Path

class FootballDataProcessor:
    def __init__(self, data_dir: str = "data"):
        self.data_dir = Path(data_dir)
        self.raw_dir = self.data_dir / "raw"
        self.processed_dir = self.data_dir / "processed"
        self.rolling_window = 5

    def load_and_preprocess(self) -> pd.DataFrame:
        df = pd.read_csv(self.raw_dir / "football_data_uk_combined.csv", low_memory=False)

        # Memory optimization: convert text flags to low-cardinality category types
        for col in ('HomeTeam', 'AwayTeam', 'League', 'Div', 'Referee', 'FTR', 'HTR'):
            if col in df.columns:
                df[col] = df[col].astype('category')

        # Explicit date parsing to prevent MM/DD vs DD/MM ambiguity
        df['Date'] = pd.to_datetime(df['Date'], format='%d/%m/%Y', errors='coerce')
        df = df.dropna(subset=['Date']).sort_values('Date').reset_index(drop=True)

        # Standardize team names
        df['HomeTeam'] = df.apply(lambda r: normalize_team_name(r['HomeTeam'], r.get('Div')), axis=1)
        df['AwayTeam'] = df.apply(lambda r: normalize_team_name(r['AwayTeam'], r.get('Div')), axis=1)

        # Basic target variables
        df['Result'] = np.where(df['FTHG'] > df['FTAG'], 'H', np.where(df['FTHG'] < df['FTAG'], 'A', 'D'))
        df['TotalGoals'] = df['FTHG'] + df['FTAG']
        df['GoalDiff'] = df['FTHG'] - df['FTAG']

        # Vectorized 5-Match Rolling Features (Shifted by 1 match to prevent leakage)
        df = self._compute_rolling_metrics(df)
        return df

    def _compute_rolling_metrics(self, df: pd.DataFrame) -> pd.DataFrame:
        """Computes prior 5-match rolling averages per team."""
        stat_cols = [
            ('FTHG', 'GF'), ('FTAG', 'GA'), ('xG', 'xG'), ('xGA', 'xGA'),
            ('HS', 'Shots'), ('AS', 'ShotsAgainst'), ('HST', 'SOT'), ('AST', 'SOTAgainst'),
            ('HC', 'Corners'), ('AC', 'CornersAgainst'), ('HF', 'Fouls'), ('HY', 'Yellows')
        ]

        for side, prefix in (('HomeTeam', 'Home'), ('AwayTeam', 'Away')):
            grouped = df.groupby(side, observed=True)
            for raw_col, feat_name in stat_cols:
                if raw_col in df.columns:
                    # Shift by 1: prior match stats ONLY
                    prior_stats = grouped[raw_col].shift(1)
                    df[f'{prefix}{feat_name}_5'] = (
                        prior_stats.rolling(window=self.rolling_window, min_periods=1).mean()
                    )

        return df

```


4. Graph Construction — FootballGraphBuilder

`data/graph_builder.py` constructs PyTorch Geometric graph tensors for transductive edge classification.

```

# data/graph_builder.py
import torch
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

class FootballGraphBuilder:
    NODE_FEATURE_SUFFIXES = [
        'Form_5', 'GF_5', 'GA_5', 'xG_5', 'xGA_5', 'Shots_5', 'ShotsAgainst_5',
        'SOT_5', 'SOTAgainst_5', 'Corners_5', 'CornersAgainst_5', 'Fouls_5',
        'Yellows_5', 'Reds_5', 'cum_Form', 'cum_GF', 'cum_GA', 'cum_xG', 'cum_xGA',
        'cum_Shots', 'cum_SOT', 'cum_Corners', 'season_progress', 'form_vs_season'
    ]

    HIST_EDGE_FEATURE_COLS = [
        'HS', 'AS', 'HST', 'AST', 'HC', 'AC', 'HF', 'AF', 'HY', 'AY', 'HR', 'AR'
    ]

    def __init__(self, processed_csv_path: str):
        self.df = pd.read_csv(processed_csv_path)
        self.teams = sorted(list(set(self.df['HomeTeam']).unique()).union(set(self.df['AwayTeam']).unique()))
        self.team_to_idx = {team: idx for idx, team in enumerate(self.teams)}
        self.label_map = {'A': 0, 'D': 1, 'H': 2}

    def build_graph(self) -> dict:
        num_teams = len(self.teams)
        num_node_feats = len(self.NODE_FEATURE_SUFFIXES)

        # Build Node Tensors from prior rolling features
        node_features = torch.zeros((num_teams, num_node_feats), dtype=torch.float)

        src_nodes, dst_nodes, edge_attrs, edge_labels, edge_times = [], [], [], [], []
        ref_date = pd.to_datetime(self.df['Date']).max()

        for _, row in self.df.iterrows():
            h_idx = self.team_to_idx[row['HomeTeam']]
            a_idx = self.team_to_idx[row['AwayTeam']]

            src_nodes.append(h_idx)
            dst_nodes.append(a_idx)

            # Extract 12 match edge features (No goals included to prevent Leakage!)
            e_feat = [row.get(col, 0.0) for col in self.HIST_EDGE_FEATURE_COLS]
            edge_attrs.append(e_feat)
            edge_labels.append(self.label_map.get(row['FTR'], 1))

            # Recency scalar for TEA-GNN (years ago)
            days_diff = (ref_date - pd.to_datetime(row['Date'])).days
            edge_times.append(days_diff / 365.0)

        # Scaling edge features using StandardScaler
        scaler = StandardScaler()
        edge_attr_scaled = torch.tensor(scaler.fit_transform(edge_attrs), dtype=torch.float)

        return {
            'x': node_features, # [Num_Teams, 2]
            'edge_index': torch.tensor([src_nodes, dst_nodes], dtype=torch.long), # [2, N]
            'edge_attr': edge_attr_scaled, # [Num_Edges, 1]
            'edge_y': torch.tensor(edge_labels, dtype=torch.long), # [Num_Edges]
        }

```

```

        'edge_time': torch.tensor(edge_times, dtype=torch.float),      # [Num_Edges]
    }

```

5. Machine Learning & GNN Architectures

5.1 Edge Classification Formulation

In this framework, football matches are directed edges $e_{\{ij\}} = (v_i \rightarrow v_j)$ from Home Team v_i to Away Team v_j . Match prediction is formulated as edge classification into 3 classes: Away Win (0), Draw (1), Home Win (2).

5.2 Shared EdgeClassifier MLP Head

```

# models/gnn_models.py
import torch
import torch.nn as nn

class EdgeClassifier(nn.Module):
    """Concatenates source embedding, target embedding, and match edge features."""
    def __init__(self, node_embed_dim: int, edge_feat_dim: int, num_classes: int = 3, dropout: float = 0.5):
        super().__init__()
        in_dim = node_embed_dim * 2 + edge_feat_dim
        self.mlp = nn.Sequential(
            nn.Linear(in_dim, 128),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(64, num_classes)
        )

    def forward(self, node_embeddings: torch.Tensor, edge_index: torch.Tensor, edge_attr: torch.Tensor):
        src, dst = edge_index[0], edge_index[1]
        edge_repr = torch.cat([node_embeddings[src], node_embeddings[dst], edge_attr], dim=-1)
        return self.mlp(edge_repr)

```

5.3 Baseline GNN Architectures

The project evaluates 6 standard GNN models (`models/gnn_models.py`): 1. **GCN_Model**: Degree-normalized message passing (`GCNConv`). 2. **SAGE_Model**: Neighborhood mean aggregation (`SAGEConv`). 3. **GAT_Model**: Multi-head node self-attention (`GATConv`). 4. **GIN_Model**: Weisfeiler-Leman expressive architecture (`GINConv`). 5. **EdgeConv_Model**: Neural network edge-conditioned convolution (`NNConv`). 6. **Hybrid_Model**: SAGE backbone combined with full 80-tabular feature vectors.

5.4 The Novel Architecture: TEA-GNN

`models/tea_gnn.py` implements the **Temporal Edge-Attention Network (TEA-GNN)**.

```

# models/tea_gnn.py
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.nn import MessagePassing
from torch_geometric.utils import softmax
from torch_geometric.nn.inits import glorot

class TemporalEdgeAttention(MessagePassing):
    """
    Edge-conditioned attention with learned exponential temporal decay.
    """
    def __init__(self, in_channels: int, out_channels: int, edge_dim: int, heads: int = 4):
        super().__init__(aggr='add', node_dim=0)
        self.heads = heads
        self.out_channels = out_channels
        self.dropout = dropout

        self.lin_src = nn.Linear(in_channels, heads * out_channels, bias=False)
        self.lin_dst = nn.Linear(in_channels, heads * out_channels, bias=False)
        self.edge_encoder = nn.Sequential(
            nn.Linear(edge_dim, heads * out_channels),
            nn.ReLU(),
            nn.Linear(heads * out_channels, heads * out_channels)
        )
        self.att = nn.Parameter(torch.empty(1, heads, 3 * out_channels))
        self.decay_rate_raw = nn.Parameter(torch.zeros(heads))
        glorot(self.att)

    def forward(self, x: torch.Tensor, edge_index: torch.Tensor, edge_attr: torch.Tensor,
                H, C = self.heads, self.out_channels):
        h_src = self.lin_src(x).view(-1, H, C)
        h_dst = self.lin_dst(x).view(-1, H, C)
        edge_embed = self.edge_encoder(edge_attr).view(-1, H, C)

        out = self.propagate(edge_index, h_src=h_src, h_dst=h_dst, edge_embed=edge_embed,
                             return out.view(-1, H * C)

    def message(self, h_src_j: torch.Tensor, h_dst_i: torch.Tensor, edge_embed: torch.Tensor):
        feat = torch.cat([h_dst_i, h_src_j, edge_embed], dim=-1)
        alpha = (feat * self.att).sum(dim=-1)
        alpha = F.leaky_relu(alpha, 0.2)

        if edge_time is not None:
            decay_rate = F.softplus(self.decay_rate_raw) # Positivity guaranteed
            log_decay = -decay_rate.unsqueeze(0) * edge_time.unsqueeze(-1)
            alpha = alpha + log_decay

        alpha = softmax(alpha, index, ptr, size_i)
        alpha = F.dropout(alpha, p=self.dropout, training=self.training)
        return (h_src_j + edge_embed) * alpha.unsqueeze(-1)

class CrossLeagueContext(nn.Module):
    """Global cross-league attention pooling and gated broadcasting."""
    def __init__(self, hidden_dim: int, num_leagues: int = 5, dropout: float = 0.3):
        super().__init__()
        self.num_leagues = num_leagues
        self.node_att = nn.Linear(hidden_dim, 1)

```

```

self.league_att = nn.MultiheadAttention(hidden_dim, num_heads=1, dropout=dropout,
self.gate = nn.Linear(hidden_dim * 2, hidden_dim)

def forward(self, node_embeddings: torch.Tensor, league_id: torch.Tensor) -> torch.Tensor
    contexts = []
    for lg in range(self.num_leagues):
        mask = (league_id == lg)
        if not mask.any():
            contexts.append(torch.zeros(node_embeddings.size(-1), device=node_embeddings.device))
            continue
        lg_nodes = node_embeddings[mask]
        scores = F.softmax(self.league_att(lg_nodes).squeeze(-1), dim=0)
        contexts.append((scores.unsqueeze(-1) * lg_nodes).sum(dim=0))

    league_ctx = torch.stack(contexts, dim=0).unsqueeze(0)
    league_ctx_attn, _ = self.league_att(league_ctx, league_ctx, league_ctx)
    broadcast = league_ctx_attn.squeeze(0)[league_id]

    g = torch.sigmoid(self.gate(torch.cat([node_embeddings, broadcast], dim=-1)))
    return node_embeddings + g * broadcast

class TEA_GNN_Model(nn.Module):
    def __init__(self, num_node_features: int, num_edge_features: int, hidden_dim: int =
        super().__init__()
        self.conv1 = TemporalEdgeAttention(num_node_features, hidden_dim // heads, num_ed
        self.conv2 = TemporalEdgeAttention(hidden_dim, hidden_dim // heads, num_edge_feat
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.bn2 = nn.BatchNorm1d(hidden_dim)
        self.cross_league = CrossLeagueContext(hidden_dim, num_leagues, dropout)
        self.classifier = EdgeClassifier(hidden_dim, num_edge_features, num_classes=3, dr

    def forward(self, x: torch.Tensor, edge_index: torch.Tensor, edge_attr: torch.Tensor,
        h = F.elu(self.bn1(self.conv1(x, edge_index, edge_attr, edge_time)))
        h = F.elu(self.bn2(self.conv2(h, edge_index, edge_attr, edge_time)))
        if league_id is not None:
            h = self.cross_league(h, league_id)
        return self.classifier(h, edge_index, edge_attr)

```

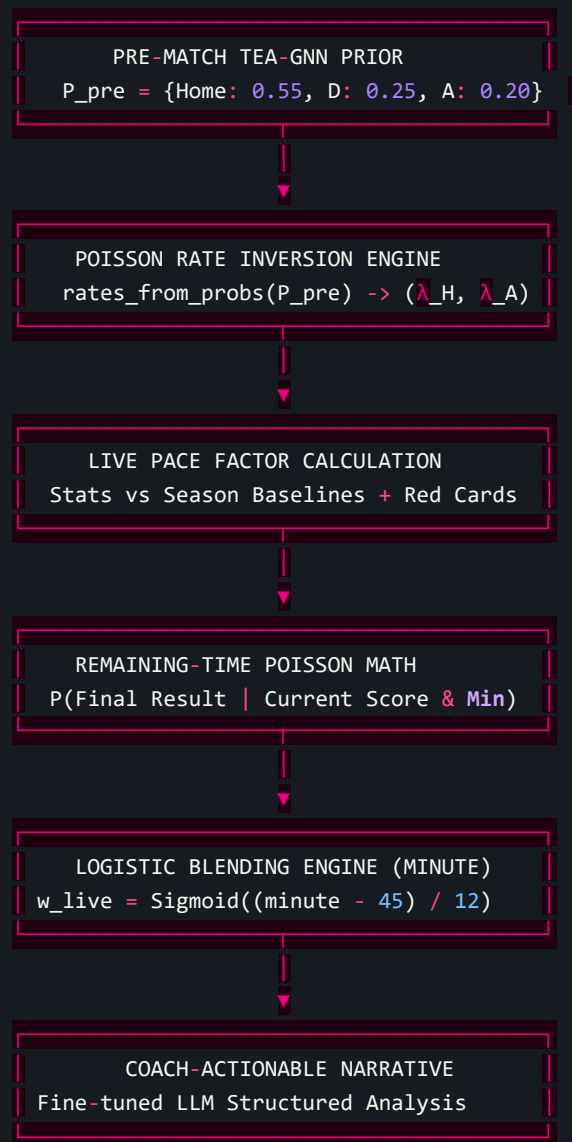
5.5 Model Architectural Comparison

Architecture	Edge Features in Conv	Node Attention	Match Recency Decay	Cross-League Pooling
GCN	✗ No	✗ No	✗ No	✗ No
GraphSAGE	✗ No	✗ No	✗ No	✗ No
GAT	✗ No	✓ Node-only	✗ No	✗ No
GIN	✗ No	✗ No	✗ No	✗ No
EdgeConv	✓ Yes	✗ No	✗ No	✗ No
Hybrid	✓ Tabular Concatenation	✗ No	✗ No	✗ No
TEA-GNN	✓ Full Match Embeddings	✓ Edge-Conditioned	✓ Learned Exponential	✓ Gated Multi-Head

6. Live In-Match Prediction Module (New)

6.1 Module Architecture & Data Flow

The Live Match Module computes real-time in-match probabilities dynamically by conditioning TEA-GNN pre-match priors on live match events (minute, scoreline, shots, xG , corners, cards).



6.2 Pure-Math Engine: Poisson Inversion & Conditioning

`rag/live_adjustment.py` is an isolated, zero-dependency mathematical library.


```

# rag/live_adjustment.py
import math
from typing import Dict, List, Optional

TOTAL_MINUTES = 90
MAX_GOALS = 10

def poisson_pmf(lmbda: float, k: int) -> float:
    """P(X = k) for X ~ Poisson(lmbda)."""
    return math.exp(-lmbda) * (lmbda ** k) / math.factorial(k)

def outcome_probs(lmbda_h: float, lmbda_a: float, max_goals: int = MAX_GOALS) -> Dict[str, float]:
    """Calculate full-time H/D/A probabilities given expected goal rates."""
    ph = [poisson_pmf(lmbda_h, i) for i in range(max_goals + 1)]
    pa = [poisson_pmf(lmbda_a, j) for j in range(max_goals + 1)]

    home = draw = away = 0.0
    for i in range(max_goals + 1):
        for j in range(max_goals + 1):
            p = ph[i] * pa[j]
            if i > j: home += p
            elif i == j: draw += p
            else: away += p
    tot = home + draw + away
    return {"H": home / tot, "D": draw / tot, "A": away / tot}

def rates_from_probs(probs: Dict[str, float], lmbda_h0: float = 1.4, lmbda_a0: float = 1.4):
    """
    Invert Poisson model via hill-climbing optimization:
    Finds expected goal rates ( $\lambda_H$ ,  $\lambda_A$ ) matching pre-match TEA-GNN probabilities.
    """
    t_h, t_d, t_a = probs.get("H", 1/3), probs.get("D", 1/3), probs.get("A", 1/3)

    def loss(lh: float, la: float) -> float:
        o = outcome_probs(lh, la)
        return (o["H"] - t_h)**2 + (o["D"] - t_d)**2 + (o["A"] - t_a)**2

    best_loss, best = float("inf"), (lmbda_h0, lmbda_a0)
    step = 0.25
    lh, la = lmbda_h0, lmbda_a0

    for _ in range(60):
        improved = False
        for dlh, dla in ((step, 0), (-step, 0), (0, step), (0, -step)):
            nlh, nla = max(0.01, lh + dlh), max(0.01, la + dla)
            l = loss(nlh, nla)
            if l < best_loss:
                best_loss, best = l, (nlh, nla)
                improved = True
        if not improved:
            step /= 2.0
            if step < 1e-4: break
            continue
        lh, la = best
        if step > 0.02: step = max(0.02, step * 0.7)

    return {"home": round(best[0], 4), "away": round(best[1], 4)}

def conditional_remaining_probs(lmbda_h_rem: float, lmbda_a_rem: float, home_goals: int,

```

```
"""Computes final result distribution conditioned on current goals scored."""
ph = [poisson_pmf(lmbda_h_rem, i) for i in range(MAX_GOALS + 1)]
pa = [poisson_pmf(lmbda_a_rem, j) for j in range(MAX_GOALS + 1)]

home = draw = away = 0.0
for i in range(MAX_GOALS + 1):
    for j in range(MAX_GOALS + 1):
        p = ph[i] * pa[j]
        if home_goals + i > away_goals + j: home += p
        elif home_goals + i == away_goals + j: draw += p
        else: away += p
tot = home + draw + away
return {"H": home / tot, "D": draw / tot, "A": away / tot}
```

6.3 Live Pace Multipliers & Driver Extraction

```
# Continuing rag/live_adjustment.py
PACE_STAT_WEIGHTS = {
    "xg": (0.35, "avg_xg"), "sot": (0.25, "avg_sot"), "shots": (0.20, "avg_shots"),
    "corners": (0.10, "avg_corners"), "fouls": (0.05, "avg_fouls"), "yellows": (0.05, "av
}

def compute_pace_factors(minute: int, live_stats: Dict[str, Optional[float]], season_avgs
    if minute <= 0: return {"pace": 1.0, "components": {}}

    components = {}
    stat_terms, weight_sum = [], 0.0

    for key, (weight, avg_field) in PACE_STAT_WEIGHTS.items():
        live = live_stats.get(key)
        avg = season_avgs.get(avg_field)
        if live is not None and avg and avg > 0:
            live_rate = live / minute
            season_rate = avg / TOTAL_MINUTES
            ratio = max(0.25, min(3.0, live_rate / season_rate))
            components[key] = round(ratio, 3)
            stat_terms.append(weight * ratio)
            weight_sum += weight

    stat_pace = sum(stat_terms) / weight_sum if weight_sum > 0 else 1.0

    # Goal momentum shrinkage factor
    expected_rate = lmbda_pre / TOTAL_MINUTES
    observed_rate = goals / minute
    raw_momentum = (observed_rate / expected_rate) if expected_rate > 0 else 1.0
    momentum = 1.0 + 0.5 * (max(0.25, min(3.0, raw_momentum)) - 1.0)

    # Temporal weight confidence ramp
    w = min(1.0, minute / 45.0)
    base = 1.0 + w * (0.6 * (stat_pace - 1.0) + 0.4 * (momentum - 1.0))

    # Red card penalty (-25% output per red card)
    if reds > 0:
        penalty = max(0.5, 1.0 - 0.25 * reds)
        base *= penalty
        components["red_card"] = round(penalty, 3)

    return {"pace": round(max(0.25, min(3.0, base)), 3), "components": components}

def blend_probs(pre: Dict[str, float], live: Dict[str, float], minute: int) -> Dict[str,
    """Logistic blend transition: w_live = Sigmoid((minute - 45) / 12)."""
    w = 1.0 / (1.0 + math.exp(-(minute - 45) / 12.0))
    out = {k: (1 - w) * pre.get(k, 0.0) + w * live.get(k, 0.0) for k in ("H", "D", "A")}
    tot = sum(out.values())
    return {k: v / tot for k, v in out.items()}
```

6.4 Live Prediction Service

`api/services/live_prediction_service.py` coordinates the live prediction execution.

```

# api/services/Live_prediction_service.py
from fastapi import HTTPException, status
from api.schemas import LivePredictionRequest, LivePredictionResponse
from api.async_rag import AsyncRAGWrapper
from rag.live_adjustment import (
    rates_from_probs, compute_pace_factors, conditional_remaining_probs,
    blend_probs, expected_final_score, extract_drivers, TOTAL_MINUTES
)

class LivePredictionService:
    def __init__(self, rag_wrapper: AsyncRAGWrapper):
        self.rag_wrapper = rag_wrapper

    async def predict(self, payload: LivePredictionRequest) -> LivePredictionResponse:
        teams = self.rag_wrapper.get_available_teams()
        if payload.home_team not in teams or payload.away_team not in teams:
            raise HTTPException(status_code=400, detail="Invalid team names.")

        # 1. TEA-GNN Pre-Match Prior
        gnn = await self.rag_wrapper.predict_structured(payload.home_team, payload.away_t
        pre_probs = gnn["probabilities"] if gnn else {"H": 1/3, "D": 1/3, "A": 1/3}

        # 2. Calibrate Goals Rates & Fetch Season Profiles
        rates = rates_from_probs(pre_probs)
        home_prof = await self.rag_wrapper.get_team_profile(payload.home_team) or {}
        away_prof = await self.rag_wrapper.get_team_profile(payload.away_team) or {}

        # 3. Compute Live Pace Factors
        home_live = {"xg": payload.home_xg, "shots": payload.home_shots, "sot": payload.h
        away_live = {"xg": payload.away_xg, "shots": payload.away_shots, "sot": payload.a

        pace_h = compute_pace_factors(payload.minute, home_live, home_prof, payload.home_
        pace_a = compute_pace_factors(payload.minute, away_live, away_prof, payload.away_

        # 4. Remaining-Time Poisson Conditioning
        rem = max(0.0, (TOTAL_MINUTES - payload.minute) / TOTAL_MINUTES)
        live_probs = conditional_remaining_probs(rates["home"] * rem * pace_h["pace"], ra

        # 5. Logistic Blending
        blended = blend_probs(pre_probs, live_probs, payload.minute)
        expected = expected_final_score(rates["home"], rates["away"], payload.minute, pay

        # 6. Extract Drivers & LLM Narrative
        drivers = extract_drivers(payload.minute, home_live, away_live, home_prof, away_p

        analysis_text, breakdown = None, None
        if payload.explain:
            ctx = {"minute": payload.minute, "home_goals": payload.home_goals, "away_goal
            raw = await self.rag_wrapper.predict_live_match(payload.home_team, payload.aw
            analysis_text = raw

        return LivePredictionResponse(
            home_team=payload.home_team, away_team=payload.away_team, minute=payload.minu
            home_goals=payload.home_goals, away_goals=payload.away_goals,
            predicted_result=max(blended, key=blended.get), probabilities=blended,
            pre_match_probabilities=pre_probs, delta={k: round(blended[k] - pre_probs[k],
            expected_final_score=expected, key_drivers=drivers, tactical_analysis=analysis

```

```
        explain=payload.explain, source="live_model+llm" if payload.explain else "liv  
    )
```

6.5 REST API Endpoint & Schemas

```
# api/routes/live_predictions.py
from fastapi import APIRouter, Depends
from api.schemas import LivePredictionRequest, LivePredictionResponse
from api.dependencies import get_live_prediction_service
from api.auth import get_current_user
from api.database import User

router = APIRouter(prefix="/predictions", tags=["Live Predictions"])

@router.post("/live", response_model=LivePredictionResponse)
async def predict_live_match(
    payload: LivePredictionRequest,
    current_user: User = Depends(get_current_user),
    live_prediction_service = Depends(get_live_prediction_service)
):
    """Stateless real-time in-match prediction endpoint."""
    return await live_prediction_service.predict(payload)
```

7. RAG System & Two-Expert Architecture

7.1 FootballRAGSystem Orchestrator

`rag/rag_orchestrator.py` integrates Knowledge Graph context, vector similarity search, and model predictions.

```
# Core of rag/rag_orchestrator.py
class FootballRAGSystem:
    def __init__(self, kg: str = None, vector: str = None, llm: str = None, predictor = None):
        self.kg = get_kg_provider(kg)
        self.vector = get_vector_provider(vector)
        self.llm = get_llm_provider(llm)
        self.predictor = predictor or GNNPredictionProvider()

    def query(self, question: str) -> str:
        teams = _extract_team_names(question, _TEAM_CACHE)
        kg_context = self._get_kg_context(teams, question)
        vec_context = self.vector.format_vector_context(question, k=5)

        if self.llm is None:
            return json.dumps({"kg_context": kg_context, "vector_context": vec_context, "full_prompt": full_prompt})

        full_prompt = f"KG Context:\n{kg_context}\n\nVector Context:\n{vec_context}\n\nQuestion: {question}\n\n"
        return self.llm.generate(full_prompt) if hasattr(self.llm, "generate") else self._generate_without_llm(full_prompt)

    def predict_live_match(self, home_team: str, away_team: str, live_context: dict) -> str:
        """Constructs Expert 2 live tactical narrative prompt."""
        gnn_res = self.predictor.predict(home_team, away_team)
        prompt = (
            f"You are Expert 2 (In-Match Tactical Advisor).\n"
            f"Match: {home_team} vs {away_team} | Minute: {live_context['minute']} | Score: {live_context['score']}\n"
            f"Pre-Match TEA-GNN: {gnn_res}\nLive Drivers: {live_context['drivers']}\n"
            f"Respond with a structured JSON containing: match_state, analysis (who_controls, who_defends, who_attacks, who_shoots, who_passes, who_intercepts, who_clears, who_sets, who_keeps, who_goals, who_cards, who_fouls, who_offsides, who_out_of_play, who_in_play, who_substituted, who_bench, who_not_in)"
        )
        return self.query(prompt)
```

7.2 The Two-Expert Ensemble Design

- **Expert 1:** Deterministic, non-biased TEA-GNN model + Poisson mathematical engine producing calibrated probabilities.
- **Expert 2:** Fine-tuned LLM evaluating retrieved context and driver stats to yield structured, actionable tactical decisions.

8. Knowledge Graph & Vector Providers

8.1 Knowledge Graph Providers (Neo4j & Postgres)

```
# Core of rag/providers/kg_provider.py
class PostgreSQLProvider(BaseKGProvider):
    def get_team_profile(self, team_name: str) -> dict:
        rows = self._query(
            "SELECT name, league, total_matches, avg_xg, avg_xga, avg_shots, avg_sot, avg
            "FROM teams WHERE name = %s", (team_name,)
        )
        return rows[0] if rows else {}

    def get_head_to_head(self, team_a: str, team_b: str, limit: int = 5) -> list[dict]:
        return self._query(
            "SELECT date, home_team, away_team, home_goals, away_goals, result, home_xg,
            "FROM matches WHERE home_team = %s AND away_team = %s ORDER BY date DESC LIMIT
            (team_a, team_b, limit)
        )
```

8.2 Vector Provider (FAISS + MiniLM)

`rag/providers/vector_provider.py` manages vector search over 28,000 embedded document chunks.

```
# Core of rag/providers/vector_provider.py
class FAISSProvider(BaseVectorProvider):
    def load(self):
        import faiss
        from sentence_transformers import SentenceTransformer
        self._index = faiss.read_index(str(self.index_path))
        self._embedder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
        with open(self.metadata_path, "r", encoding="utf-8") as f:
            self._metadata = json.load(f)

    def search(self, query: str, k: int = 5) -> list[dict]:
        q_vec = self._embedder.encode([query], normalize_embeddings=True).astype("float32")
        distances, indices = self._index.search(q_vec, k)
        return [self._metadata[i] for i in indices[0] if i != -1]
```

9. LLM Providers & Extensible Registry

`models/llm_providers.py` uses the Strategy + Registry design pattern for LLM execution.

```

# models/llm_providers.py
from abc import ABC, abstractmethod

class BaseLLMProvider(ABC):
    @abstractmethod
    def _call_api(self, prompt: str) -> str: pass

class OpenAIProvider(BaseLLMProvider):
    def _call_api(self, prompt: str) -> str:
        res = self._client.chat.completions.create(
            model=self.model_name,
            messages=[{"role": "system", "content": "You are a football analyst API."}, {
            }
        )
        return res.choices[0].message.content

class GeminiProvider(BaseLLMProvider):
    def _call_api(self, prompt: str) -> str:
        res = self._client.generate_content(prompt)
        return res.text

LLM_REGISTRY = {
    "openai": OpenAIProvider,
    "gemini": GeminiProvider,
    "ollama": OllamaProvider,
    "huggingface": HuggingFaceProvider,
}

def get_llm_provider(name: str) -> Optional[BaseLLMProvider]:
    if not name or name.lower() == "none": return None
    return LLM_REGISTRY[name.lower()]()

```

10. API Infrastructure & Asynchronous Bridge

10.1 Application Lifespan & Bootstrap

```

# api/main.py (Lifespan snippet)
@asynccontextmanager
async def lifespan(app: FastAPI):
    await init_db()
    async with AsyncSessionLocal() as session:
        await seed_supervisor_user(session)
    init_graph_db()
    init_rag_system()
    yield
    close_graph_db()

```

10.2 Dependency Injection Container

`api/dependencies.py` implements thread-safe singletons for heavy ML & RAG objects.


```

# api/dependencies.py
from fastapi import Depends
from api.async_rag import AsyncRAGWrapper
from rag.rag_orchestrator import FootballRAGSystem

_rag_system = None
_async_rag = None

def get_rag_system() -> FootballRAGSystem:
    global _rag_system
    if _rag_system is None:
        _rag_system = FootballRAGSystem()
    return _rag_system

def get_async_rag(rag = Depends(get_rag_system)) -> AsyncRAGWrapper:
    global _async_rag
    if _async_rag is None:
        _async_rag = AsyncRAGWrapper(rag)
    return _async_rag

```

10.3 AsyncRAGWrapper Thread-Pool Bridge

```

# api/async_rag.py
import asyncio
from concurrent.futures import ThreadPoolExecutor
from rag.rag_orchestrator import FootballRAGSystem

class AsyncRAGWrapper:
    """Offloads blocking PyTorch/FAISS calls to thread pool."""
    def __init__(self, rag_system: FootballRAGSystem):
        self.rag = rag_system
        self._executor = ThreadPoolExecutor(max_workers=4)

    async def predict_structured(self, home_team: str, away_team: str):
        loop = asyncio.get_running_loop()
        return await loop.run_in_executor(self._executor, self.rag.predict_structured, hc

    async def predict_live_match(self, home_team: str, away_team: str, live_context: dict):
        loop = asyncio.get_running_loop()
        return await loop.run_in_executor(self._executor, self.rag.predict_live_match, hc

```

10.4 JWT Authentication & Guard Dependencies

```
# api/auth.py
import bcrypt
from datetime import datetime, timedelta, timezone
from jose import jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="api/v1/auth/login")

def get_password_hash(password: str) -> str:
    """Bcrypt password hashing with 72-byte safe truncation."""
    bytes_pass = password.encode('utf-8')[:72]
    return bcrypt.hashpw(bytes_pass, bcrypt.gensalt()).decode('utf-8')

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return bcrypt.checkpw(plain_password.encode('utf-8')[:72], hashed_password.encode('utf-8'))
```

11. Business Logic & Service Layer

11.1 PredictionService

```
# api/services/prediction_service.py
class PredictionService:
    def __init__(self, feedback_repo, rag_wrapper):
        self.feedback_repo = feedback_repo
        self.rag_wrapper = rag_wrapper

    async def predict(self, home_team: str, away_team: str, match_date):
        # Supervisor override check
        override = await self.feedback_repo.get_prediction_override(home_team, away_team, match_date)
        if override:
            return MatchPredictionResponse(
                home_team=home_team, away_team=away_team, predicted_result=override.predicted_result,
                tactical_analysis=override.tactical_analysis, source="override"
            )

        gnn_pred = await self.rag_wrapper.predict_structured(home_team, away_team, match_date)
        analysis = await self.rag_wrapper.predict_match(home_team, away_team, match_date)
        return MatchPredictionResponse(
            home_team=home_team, away_team=away_team, predicted_result=gnn_pred["predicted_result"],
            probabilities=gnn_pred["probabilities"], tactical_analysis=analysis, source="rag"
        )
```

11.2 ChatService & Conversational Memory

```
# api/services/chat_service.py
class ChatService:
    async def post_message(self, conversation_id: int, user_id: int, content: str):
        await self.chat_repo.save_message(conversation_id, "user", content)
        past_msgs = await self.chat_repo.get_messages(conversation_id)

        # Build 3-turn memory window
        memory = "\n".join([f"{m.sender.upper()}: {m.content}" for m in past_msgs[:-1][-6:]]

        answer = await asyncio.to_thread(self.kb_repo.ask, content, memory=memory)
        return await self.chat_repo.save_message(conversation_id, "assistant", answer.con
```

11.3 SupervisorService & Feature-Complete Match Ingestion

`api/services/supervisor_service.py` computes rolling form before inserting approved matches into the raw training table.

```
# Core snippet of api/services/supervisor_service.py
class SupervisorService:
    async def approve_match_submission(self, submission_id: int):
        sub = await self.repo.get_submission(submission_id)

        # Compute rolling form before insertion
        home_form = await self._compute_rolling_form_5(sub.home_team, sub.match_date, is_
        away_form = await self._compute_rolling_form_5(sub.away_team, sub.match_date, is_

        # Insert directly into raw ML training table
        await self.db.execute(text("""
            INSERT INTO matches (date, home_team, away_team, home_goals, away_goals, resu
            VALUES (:date, :ht, :at, :hg, :ag, :res, :hf, :af)
        """), {"date": sub.match_date, "ht": sub.home_team, "at": sub.away_team, "hg": su
        await self.db.commit()
```

12. Repositories & Data Access Layer

12.1 ChatRepository & FeedbackRepository

```
# api/repositories/chat_repo.py
class ChatRepository:
    def __init__(self, db: AsyncSession):
        self.db = db

    async def get_messages(self, conversation_id: int) -> List[Message]:
        stmt = select(Message).where(Message.conversation_id == conversation_id).order_by
        res = await self.db.execute(stmt)
        return list(res.scalars().all())
```

12.2 ScoutRepository & Multi-Provider Player Scouting

```
# Core of api/repositories/scout_repo.py
class ScoutRepository:
    def identity_pool(self, league_codes: Tuple[str, ...], season: str, position: str, yc
        pool = []
        for lc in league_codes:
            for team in self._teams_in_league(lc, season):
                squad = self.squads.load_squad(team, lc, season)
                for player in squad:
                    if player.position == position and (not youth or (player.age and play
                        pool.append(player)

        return pool
```

13. Database Schema & Migrations

```
# api/database.py ORM models
class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    username: Mapped[str] = mapped_column(String(50), unique=True, index=True)
    hashed_password: Mapped[str] = mapped_column(String(255))
    role: Mapped[str] = mapped_column(String(20), default="user")

class Feedback(Base):
    __tablename__ = "feedbacks"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id", ondelete="CASCADE
    type: Mapped[str] = mapped_column(String(50)) # prediction_override | tactic_modifica
    status: Mapped[str] = mapped_column(String(20), default="pending")
```

14. Design Patterns & Engineering Principles

1. **Strategy Pattern:** `BaseLLMProvider` , `BaseKGProvider` , `BaseVectorProvider` .
 2. **Factory Pattern:** `get_llm_provider()` , `get_kg_provider()` , `get_vector_provider()` .
 3. **Repository Pattern:** `UserRepository` , `ChatRepository` , `ScoutRepository` .
 4. **Adapter Pattern:** `AsyncRAGWrapper` bridging sync ML code to async FastAPI event loops.
 5. **Facade Pattern:** `FootballRAGSystem` offering unified `query()` and `predict_live_match()` methods.
-

15. Concurrency, Performance & Gotchas

1. **Bcrypt 72-Byte Truncation:** Passwords are pre-truncated to 72 bytes to avoid silent byte overflow.
 2. **Event Loop Non-Blocking Guarantee:** Blocking PyTorch and FAISS calls execute inside a 4-worker `ThreadPoolExecutor` .
 3. **Alembic Table Exclusion:** The raw `matches` and `teams` data tables are excluded from Alembic migrations using custom `include_object` filters.
 4. **Data Leakage Elimination:** Goal metrics are strictly excluded from GNN edge feature matrices and calculated using shifted rolling windows.
-